dbt test 完全ガイド

Snowflakeユーザーのためのデータ品質テスト

作成者: 菊池翼

今日話すこと

- dbt testとは何か
- テストの3つのカテゴリ：Generic tests と Singular tests と Unit tests
- 組み込みGeneric tests（4種）の紹介
- Singular testsの紹介
- Unit testsの紹介
- dbt_utils パッケージによるテスト拡張
- テスト設定（severity、store_failures）
- Snowflake固有の考慮事項
- ベストプラクティス

本日の例で使うデータモデル

ECサイトを題材にした3つのモデルを使います。

- `stg_customers` — 顧客マスタ（customer_id, name, email, created_at）
- `stg_orders` — 注文データ（order_id, customer_id, status, order_date, amount）
- `stg_products` — 商品マスタ（product_id, product_name, category, price）

dbt testとは何か

dbt testとは何か

データに対する「アサーション（表明）」。

テストの本質はとてもシンプル：

> **「失敗する行を返すSELECT文」**

- 0行返る → テストパス（問題なし）
- 1行以上返る → テストフェイル（問題あり）

なぜSnowflakeユーザーにdbt testが必要か

Snowflakeでは制約（PRIMARY KEY, FOREIGN KEY, UNIQUE）を定義できるが、NOT NULL以外は **強制されない** 。

```
-- Snowflakeでこう定義しても...
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT REFERENCES customers(customer_id)
);

-- 重複したorder_idをINSERTしてもエラーにならない！
INSERT INTO orders VALUES (1, 100), (1, 200);  -- 成功してしまう
```

dbt testがこのギャップを埋める。

テストの3つのカテゴリ（1/2）

Generic tests（汎用テスト）

- YAMLで宣言的に定義する
- 再利用可能で、パラメータを渡せる
- dbt組み込みの4種 + パッケージで拡張可能

Singular tests（個別テスト）

- `tests/` ディレクトリにSQLファイルとして配置する
- プロジェクト固有の一回限りの検証に使う
- 自由にSQLを書ける

テストの3つのカテゴリ（2/2）

Unit tests（単体テスト、dbt v1.8〜）

- モックデータを使ってSQLロジックを検証する
- モデル構築の前に実行される
- 本番データに依存しない

組み込みGeneric tests

組み込みGeneric tests — 概要

dbtには4つのGeneric testが組み込まれている。

テスト	検証内容
`unique`	カラムの値に重複がないか
`not_null`	カラムにNULLが含まれていないか
`accepted_values`	カラムの値が指定リストに含まれるか
`relationships`	カラムの値が別テーブルに存在するか（参照整合性）

いずれもYAMLで宣言するだけで、dbtがSQLを自動生成して実行する。

組み込みGeneric test ①: unique

カラムの全値がユニーク（重複なし）であることを検証する。

YAMLでの定義

```yaml
columns:
  - name: order_id
    data_tests:
      - unique
```

dbtが生成する実際のSQL

```sql
SELECT
  order_id AS unique_field,
  COUNT(*) AS n_records
FROM analytics.stg_orders
WHERE order_id IS NOT NULL
GROUP BY order_id
HAVING COUNT(*) > 1
```

主キーの検証に必須。Snowflakeが強制しないUNIQUE制約の代わりになる。

組み込みGeneric test ②: not_null

カラムにNULL値が存在しないことを検証する。

YAMLでの定義

```yaml
columns:
  - name: customer_id
    data_tests:
      - not_null
```

dbtが生成する実際のSQL

```sql
SELECT customer_id
FROM analytics.stg_orders
WHERE customer_id IS NULL
```

Snowflakeでは NOT NULL 制約は強制されるが、ソースデータやSTAGINGモデルでは保証されないため、テストが有効。

組み込みGeneric test ③: accepted_values

カラムの値が指定されたリストに含まれることを検証する。

YAMLでの定義

```yaml
columns:
  - name: status
    data_tests:
      - accepted_values:
          values:
            - 'placed'
            - 'shipped'
            - 'completed'
            - 'returned'
```

dbtが生成する実際のSQL

```sql
SELECT status AS value_field,
       COUNT(*) AS n_records
FROM analytics.stg_orders
WHERE status NOT IN (
  'placed', 'shipped',
  'completed', 'returned'
)
GROUP BY status
```

上流システムの変更で想定外の値が入ったとき、すぐに検知できる。

組み込みGeneric test ④: relationships

あるカラムの全値が、別テーブルの参照カラムに存在することを検証する（参照整合性チェック）。

YAMLでの定義

```yaml
columns:
  - name: customer_id
    data_tests:
      - relationships:
          to: ref('stg_customers')
          field: customer_id
```

dbtが生成する実際のSQL

```sql
SELECT o.customer_id AS id
FROM analytics.stg_orders AS o
LEFT JOIN analytics.stg_customers AS c
  ON o.customer_id = c.customer_id
WHERE o.customer_id IS NOT NULL
  AND c.customer_id IS NULL
```

Snowflakeが強制しないFOREIGN KEY制約の実質的な代替。

Singular tests

Singular tests — 概要

`tests/` ディレクトリにSQLファイルを置くだけで、テストとして認識される。

特徴：

- YAMLの宣言ではなく、自由にSQLを記述する
- 複数テーブルをJOINした複雑な検証ができる
- プロジェクト固有のビジネスルールの検証に向いている
- 「結果が0行ならパス」というルールはGeneric testと同じ

Singular tests — 例①: 金額の整合性チェック

```sql
-- tests/assert_positive_order_amounts.sql
-- 注文金額がマイナスになっていないことを確認する
SELECT
  order_id,
  amount
FROM {{ ref('stg_orders') }}
WHERE amount < 0
```

シンプルな条件チェック。Generic testの `expression_is_true` でも書けるが、SQLファイルとして管理したい場合に使う。

Singular tests — 例②: テーブル間の整合性チェック

```sql
-- tests/assert_order_amount_matches_products.sql
-- 注文金額が商品の合計金額と一致することを確認する
SELECT
  o.order_id,
  o.amount AS order_amount,
  SUM(p.price) AS product_total
FROM {{ ref('stg_orders') }} AS o
JOIN {{ ref('order_items') }} AS oi
  ON o.order_id = oi.order_id
JOIN {{ ref('stg_products') }} AS p
  ON oi.product_id = p.product_id
GROUP BY o.order_id, o.amount
HAVING o.amount != SUM(p.price)
```

複数テーブルをJOINする複雑なビジネスルールは、Singular testが適している。

Unit tests（dbt v1.8～）

Unit tests — Data tests と Unit tests の違い

	Data tests	Unit tests
いつ実行？	モデル構築の後（実データに対して）	モデル構築の前（モックデータに対して）
何をテスト？	データの品質・整合性	SQL変換ロジックの正しさ
入力データ	Snowflake上の実テーブル	YAMLで定義する静的データ
コスト	Snowflakeのクレジットを消費	最小限（静的データのみ）
使う場面	「データは正しいか？」	「コードは正しいか？」

Unit tests — なぜ必要か

Data testsだけでは検出が難しいケース：

- まだ本番に存在しないエッジケース
- 複雑なCASE文の分岐漏れ
- ウィンドウ関数の境界条件
- リファクタリング時の回帰テスト

Unit testsは **本番データに依存せず** にSQLロジックを検証できる。

Unit tests — テスト対象のモデル例

注文データからステータスラベルと税込金額を計算するモデルを考える。

```sql
-- models/fct_orders.sql
SELECT
  order_id,
  customer_id,
  status,
  CASE
    WHEN status = 'placed'    THEN '注文済み'
    WHEN status = 'shipped'   THEN '発送済み'
    WHEN status = 'completed' THEN '完了'
    WHEN status = 'returned'  THEN '返品'
    ELSE '不明'
  END AS status_label,
  amount,
  ROUND(amount * 1.1, 2) AS amount_with_tax,
  order_date
FROM {{ ref('stg_orders') }}
```

Unit tests — テストの定義

```yaml
unit_tests:
  - name: test_status_label_mapping
    description: "ステータスラベルの変換が正しいことを確認"
    model: fct_orders
    given:
      - input: ref('stg_orders')
        rows:
          - {order_id: 1, customer_id: 100, status: "placed",    amount: 1000, order_date: "2024-01-01"}
          - {order_id: 2, customer_id: 101, status: "shipped",   amount: 2000, order_date: "2024-01-02"}
          - {order_id: 3, customer_id: 102, status: "completed", amount: 3000, order_date: "2024-01-03"}
          - {order_id: 4, customer_id: 103, status: "returned",  amount: 4000, order_date: "2024-01-04"}
          - {order_id: 5, customer_id: 104, status: "unknown",   amount: 5000, order_date: "2024-01-05"}
    expect:
      rows:
        - {order_id: 1, status_label: "注文済み"}
        - {order_id: 2, status_label: "発送済み"}
        - {order_id: 3, status_label: "完了"}
        - {order_id: 4, status_label: "返品"}
        - {order_id: 5, status_label: "不明"}
```

Unit tests — 税込計算のテスト

```yaml
unit_tests:
  - name: test_amount_with_tax_calculation
    description: "税込金額の計算が正しいことを確認（10%税率）"
    model: fct_orders
    given:
      - input: ref('stg_orders')
        rows:
          - {order_id: 1, customer_id: 100, status: "placed", amount: 1000,  order_date: "2024-01-01"}
          - {order_id: 2, customer_id: 101, status: "placed", amount: 999,   order_date: "2024-01-02"}
          - {order_id: 3, customer_id: 102, status: "placed", amount: 0,     order_date: "2024-01-03"}
    expect:
      rows:
        - {order_id: 1, amount_with_tax: 1100.00}
        - {order_id: 2, amount_with_tax: 1098.90}
        - {order_id: 3, amount_with_tax: 0.00}
```

エッジケース（端数、ゼロ）もモックデータで簡単に検証できる。

Unit tests — 実行方法

```
# Unit testsのみ実行
dbt test --select test_type:unit

# 特定のUnit test
dbt test --select test_name:test_status_label_mapping

# Data testsとUnit testsを両方実行
dbt test
```

Unit testsは `dbt build` の中でモデルの構築前に実行される。ロジックに問題があればモデルの構築自体がスキップされる。

dbt_utils パッケージ

テストパッケージ: dbt_utils

dbt_utilsをインストールすると、便利なGeneric testが追加される。

```yaml
# packages.yml
packages:
  - package: dbt-labs/dbt_utils
    version: 1.3.3
```

```
dbt deps
```

dbt_utils — unique_combination_of_columns

複数カラムの組み合わせでユニークであることを検証する。

```yaml
models:
  - name: stg_orders
    data_tests:
      - dbt_utils.unique_combination_of_columns:
          combination_of_columns:
            - order_id
            - order_date
```

複合主キーや、ファクトテーブルのグレイン（粒度）の検証に使う。

dbt_utils — expression_is_true

任意のSQL式が全行でTRUEであることを検証する。

```
models:
  - name: stg_orders
    data_tests:
      - dbt_utils.expression_is_true:
          expression: "amount >= 0"
      - dbt_utils.expression_is_true:
          expression: "order_date <= CURRENT_DATE()"
```

カスタムGeneric testを書くほどでもない単純な条件チェックに便利。

dbt_utils — recency

テーブルに最近のデータが存在することを検証する。

```yaml
models:
  - name: stg_orders
    data_tests:
      - dbt_utils.recency:
          datepart: day
          field: order_date
          interval: 1
```

「最後の注文が1日以内にあること」を検証し、データ取り込みの遅延を検知する。

dbt_utils — その他の主要なテスト

テスト名	用途
`equal_rowcount`	2テーブルの行数が一致するか
`fewer_rows_than`	あるテーブルの行数が別のテーブル以下か
`equality`	2テーブルの内容が完全に一致するか
`accepted_range`	値が指定範囲内（min/max）か
`not_constant`	カラムが全行同じ値でないか
`not_empty_string`	空文字列が含まれていないか
`relationships_where`	条件付きの参照整合性チェック
`at_least_one`	最低1つの非NULLの値があるか

テスト設定

dbt testコマンド

```
# 全テスト実行
dbt test

# 特定モデルのテストのみ
dbt test --select stg_orders

# カテゴリ別に実行
dbt test --select test_type:generic
dbt test --select test_type:singular
dbt test --select test_type:unit

# タグで絞り込み
dbt test --select tag:critical

# 失敗行をテーブルに保存
dbt test --store-failures
```

テスト設定 — severity（重要度）

テスト失敗時の挙動を制御する。

error（デフォルト）

テスト失敗でパイプライン停止（exit code 1）。CI/CDでブロックされる。

warn

テスト失敗で警告を出すが、パイプラインは止まらない。

```yaml
columns:
  - name: email
    data_tests:
      - not_null:
          config:
            severity: warn
```

テスト設定 — 閾値による制御

失敗行数に応じて挙動を変えられる。

```yaml
columns:
  - name: order_id
    data_tests:
      - unique:
          config:
            severity: error
            warn_if: ">10"
            error_if: ">1000"
```

- 10行以下の重複 → パス
- 11〜1000行の重複 → 警告（warn）
- 1001行以上の重複 → エラー（error）

テスト設定 — store_failures

テスト失敗した行をSnowflakeのテーブルに保存し、デバッグを容易にする。

設定方法（per-test）：

```
data_tests:
  - unique:
      config:
        store_failures: true
        store_failures_as: table
```

プロジェクト全体で設定（dbt_project.yml）：

```
data_tests:
  +store_failures: true
```

テスト設定 — store_failures（確認方法）

失敗行の保存先: `<schema>_dbt_test__audit` スキーマにテーブルが作成される。

```
-- 失敗行を確認する
SELECT * FROM analytics_dbt_test__audit.unique_stg_orders_order_id;
```

Snowflake固有の考慮事項

Snowflake固有の考慮事項

テスト = SQLクエリ = クレジット消費

全てのdbt testはSnowflake上でSELECTクエリとして実行される。テスト数が多いほど、ウェアハウスのコストが増える。

コスト最適化のポイント

- テスト専用に小さいウェアハウス（X-SMALL）を用意する
- `dbt test --select tag:critical` で重要なテストだけ毎日実行し、全テストは週次にする
- `--select` で必要なテストだけ実行する習慣をつける

Snowflake固有の考慮事項 — デバッグ

dbtが生成したSQLは `target/compiled/` ディレクトリに出力される。このSQLをSnowflakeのワークシートに貼り付けて直接実行し、失敗原因を調査できる。

Snowflake固有の考慮事項 — ネイティブdbt実行

Snowflakeにはdbtプロジェクトをネイティブに実行する機能がある。

```
-- Snowflake上でdbt testを直接実行
EXECUTE DBT PROJECT my_database.my_schema.my_dbt_project
  ARGS = 'test --target prod';
```

Snowflake Tasksと組み合わせることで、スケジュール実行も可能。

ベストプラクティス

ベストプラクティス

必須テスト

全モデルの主キーに `unique` + `not_null` を設定する。これが最低限のテストカバレッジ。

レイヤーごとのテスト戦略

レイヤー	テスト内容
Source	ソースデータの鮮度チェック、想定外の値の検知
Staging	ビジネス上の異常値、NULLチェック
Marts	主キーの検証、粒度の変更確認、集計ロジック（Unit test）

ベストプラクティス（続き）

CI/CDでのテスト実行

PRごとにテストを実行し、失敗するPRはマージしない。

severityの使い分け

- `error` — 絶対に違反してはならない制約（主キーの重複など）
- `warn` — 把握しておきたいが、パイプラインは止めたくない問題

store_failuresの活用

開発中は `store_failures: true` を有効にしておくと、失敗行を直接クエリしてデバッグできる。

ベストプラクティス（続き2）

テストにタグをつける

```
data_tests:
  - unique:
      config:
        tags: ['critical']
```

優先度別にテストを管理し、実行スケジュールを分けられる。

まとめ

- dbt testは「失敗行を返すSELECT文」というシンプルな仕組み
- 3つのカテゴリ：Generic tests、Singular tests、Unit tests
- 組み込み4種（unique, not_null, accepted_values, relationships）で基本的なデータ品質をカバー
- Singular testsで複雑なビジネスルールを自由に検証
- Unit tests（v1.8〜）でSQLロジックを本番データなしで検証できる
- dbt_utilsで実用的なテストを拡張できる
- Snowflakeが強制しない制約を、dbt testが補完する
- テストはコスト（クレジット）とのバランスを意識して運用する

参考リンク

- dbt Data Tests: https://docs.getdbt.com/docs/build/data-tests
- dbt Unit Tests: https://docs.getdbt.com/docs/build/unit-tests
- dbt_utils: https://hub.getdbt.com/dbt-labs/dbt_utils/latest/
- dbt test コマンド: https://docs.getdbt.com/reference/commands/test